STUDENT HANDBOOK

Organic Factory

The AI organic video system. How it works and what to expect from it.

A folder you open Claude Code inside of. Claude writes every Seedance prompt, checks the clips, and builds the finished video. You make the calls and you post.

Getting started is two steps

Not a file you upload into Claude, a project folder you point Claude Code at. **One:** Claude app, **Code** tab, **Local**, **Select folder**, pick `organic-factory`. **Two:** type `/start`. If it is not in your `/` menu, wrong folder.

What it does for you

- Writes the full Seedance prompt, your style identical every time
- Generates your branding, and the clips if you want
- Checks the clips itself and says what it found
- **Builds the finished video**, cut to the original, 720p 60fps
- Writes your captions and hashtags, and remembers what you already made

What it does NOT do

- **It will not post behind your back.** Buffer is the scheduler and you switch it on
- It does not pick your product or concept
- It does not have the final say on a clip. You do
- It does not spend credits on its own. Only `/brand` and `/gen` cost anything

Not until you say so. It runs unattended once you hand the gates over, page 4.

The commands, always in this order

COMMAND	WHAT IT DOES	WHERE IT STOPS
`/start`	Sets you up and explains the rest	Day one only
`/product`	Scores the product on Viral DNA, creates the folder	You approve it
`/brand`	Name, logo, box, then locks them **(credits)**	You say "locked"
`/concept`	Reads the competitor video, plans the clips	You approve the plan
`/prompts`	Writes every prompt to a file. Free	You copy them out
`/gen`	Generates the clips **(credits)**. Optional	You confirm the price
`/review`	Checks the clips, verdict with reasons	You approve or reject
`/edit`	Builds the finished, uploadable video	You watch it once
`/post`	Caption, hashtags, hook text	You approve the text
`/ship`	Schedules it to your Buffer channels	**You say go**
`/auto`	Runs the whole chain unattended	Only the gates you kept

How one video gets made

Eight steps. Each one shows who does it.

1 **You find a product** YOU

Scrolling TikTok, burner accounts, spy tools. Claude then scores it on six things: does it speak to a clear tribe, is it new to look at, would you gift it, does it prove itself in one shot, is there a reason to send it to a friend. Under 4 out of 6 it tells you straight.

2 **You build the branding** CLAUDE YOU APPROVE

This is the only phase where images get made. Name, logo, retail box, store if you want it. When you say "locked", Claude saves them as references with their own ID. From then on the product stays 1:1 with them in every later generation and is never drawn from memory.

3 **You pick the viral you are rebuilding** YOU

The whole bet of this system is **proven format, new content.** You are not inventing a format, you are taking one that already runs and putting your product in it. Hand the video to Claude.

4 **Claude works out the rhythm and plans the clips** CLAUDE

It goes through the original and finds the exact time of every cut. Out of that comes the plan: how many clips, how long each one is, what type, which references get attached. The clip durations are not guesses, they are the real segment lengths from the original. That is why the edit lines up later.

5 **Prompts and generation** CLAUDE WRITES YOU RUN

Claude writes a complete English prompt for every clip: camera, location, the character described in text, action second by second, voice, sounds with timestamps, critical rules and the do-not block. The prompts go into files. Either you run them yourself or you use `/gen`.

6 **Claude checks the clips** CLAUDE YOU DECIDE

It probes every clip for length, frozen tails and dead frames, then pulls real frames out and looks at them: is the product still identical to your locked reference, is there text burned in that should not be, did the camera move when it was told to hold. You get a verdict per clip with a reason. Your eye still wins, and whatever you reject gets written into the rules so the next video does not repeat it.

7 **The finished video** CLAUDE

Claude cuts every clip to the original's exact times, lays the original soundtrack underneath and exports 720p 60 fps, ready to upload. If a clip is shorter than the slot it has to fill it refuses to build and tells you which one to regenerate, because that hole cannot be fixed later. CapCut is optional now, only if you want to nudge something. **No text in CapCut either way.**

8 **Posting** YOU DECIDE CLAUDE SCHEDULES

Two ways. `/ship` schedules the file to your Buffer channels, staggered across the day, landing as drafts, and it shows you every post before it sends anything. Or do it by hand: the file already has the text and the sound in it, so it is one upload per platform with nothing left to add.

Three things that will quietly break it

1. Clip file names

Clips have to be `clip-1.mp4`, `clip-2.mp4`, `clip-3.mp4` in the `clips/` folder. The edit maps clip N onto segment N by the number in the name. A clip downloaded as `download (3).mp4` lands in the wrong place and **nothing warns you**. This is the most common way the whole thing looks broken.

2. The trust dialog on first launch

The first time you start Claude Code in the folder, a window asks whether you trust it. Accept it. If you skip it, the system throws away its permissions and asks you before every single step. One click, and easy to miss, because everything still appears to work.

3. The folder name includes the date

`/product` creates a folder like `2026-09-02-magnetic-lashes`. That whole string, date included, is what you pass to every later command. Without the date it will tell you the folder does not exist.

Style rules that always apply

They load automatically, you never copy them anywhere. They are what keeps it looking like a phone video instead of AI:

- **iPhone 12 realism**, fine grain, natural daylight, and always say where the light comes from
- **No bokeh, nothing cinematic, no colour grading.** Deep focus
- **No slow motion**, but ultra smooth animation. Camera static, except selfie hooks
- **Voices described like humans**, who is talking and to whom. This makes the biggest difference
- **Product 1:1 with the reference.** No text and no music inside the clip, those come later
- **Characters are never attached as an image**, they are always described in the prompt

When a generation fails

Either run it again on the same prompt, because Seedance has real variance and the second take often lands. Or tell Claude what is wrong and it rewrites it. **Always shorter, never longer.** An overstuffed motion description is the most common cause of a bad generation. One precise sentence naming the actual objects beats four vague ones. When it will not work, cut, do not add.

One more file: `rules.md`

You write that one, not Claude. When you reject something, add a line saying what to avoid and why. When something is good, add what to prefer. It loads into every session. The rest of the system only makes output faster, this is the only part that makes it **better**. Full setup is in `SETUP.md` in the folder.

Turning on autopilot

It can run the whole chain on its own, and Buffer is the scheduler that puts the finished videos out. But every gate in this system exists because something used to go wrong there. Autopilot is not a switch you flip on day one, it is what you hand over once a gate has stopped catching anything.

First, stop the per-step prompts. In the Claude app, the permission mode selector sits next to the send button. On **Manual** it asks before every action. Set it to **Accept edits** or **Auto** and one video goes from forty interruptions to about four. Do this before you touch anything else, it is most of the friction.

Then hand over the gates, one at a time. Open `autopilot.json` in the folder. Everything starts `false`. Set `enabled` to true, then turn on one gate at a time, in this order, each only after it has gone a few videos without catching a real problem:

GATE	TURN IT OVER WHEN
`plan`	First. Once `rules.md` has content, the clip plans stop needing changes
`clips`	Next. The review still runs and still rerolls, you just stop waiting for it
`publish`	When you trust everything above it
`brand`, `product`	Last. They set the direction of everything downstream

Buffer is your scheduler. With `publish_mode` on `"draft"`, finished videos stack up in Buffer for you to release. Change it to `"queue"` and they go into your Buffer queue at staggered times and post themselves. That is the actual autopost, and it is one word in one file.

Set these two before you walk away. `max_credits_per_video` is a hard ceiling and the run stops mid-video rather than cross it. `stop_after_consecutive_rejections` ends the run if you rejected the last two, because at that point the settings are wrong and a third unattended video only burns credits.

Then `/auto <product>` runs the chain. It prints what it is about to do and what it may spend before it starts.

Two things people get wrong here

Scheduled tasks in the Claude app can run this on a timer, but a scheduled task whose prompt is literally `/auto` will not fire, because credit-spending commands are blocked from starting themselves. Describe the run in the prompt instead.

And the real one: **autopilot with an empty `rules.md` gives you a lot of mediocre.** That file is the only thing that makes unattended output resemble what you would have approved. Ten videos you actually reviewed are worth more than fifty you did not.

The hook text is your cheapest test

The text is burned in at the very end, onto the already-built video. Changing it costs nothing and generates nothing: same clips, same cut, new text, re-burn. So test it. Five versions of a hook cost you five minutes, where five versions of a video cost you credits.

Ask Claude to change it and it re-runs the burn. The knobs, by name:

OPTION	WHAT IT CHANGES	DEFAULT
`--top`	How far down the text sits, as a percent of height	22, upper third
`--size`	Font size in pixels	46
`--seconds`	Text drops away after this many seconds	stays the whole video
`--plain`	Shadow instead of the dark rounded box	box is on

Why the default sits high. TikTok and Reels lay their own caption, buttons and username over the bottom third and the right edge. Text in the middle looks fine in your file and gets covered on the phone. If you move it, check it on an actual phone, not in the preview.

What makes a hook work, in the order that matters: under four words wherever it can be, said the way your tribe says it rather than the way an ad would, and legible on mute. Most people watch that first second with the sound off, so the text is doing the whole job right then.

When one works, write it down. Add it to `rules.md` as a Prefer line with the reason. Change one thing at a time or you will not know which change did it.